

TABLE OF CONTENTS

1. INTRODUCTION.....	3
2. MOTIVATION AND NEED	3
3. OBJECTIVES	3
4. SCOPE	4
5. EXISTING SYSTEM	4
6. IMPLEMENTED SYSTEM AND KEY FEATURES.....	5
7. TARGET USERS	5
8. FUNCTIONAL REQUIREMENTS	6
9. NON-FUNCTIONAL REQUIREMENTS	6
10. SYSTEM ARCHITECTURE.....	6
11. TECHNOLOGY STACK.....	7
12. DATA DESIGN	8
13. CLASS AND MODULE DESIGN	9
14. CONSOLE INTERFACE AND MENU NAVIGATION.....	9
15. FACILITY RECORD MANAGEMENT	10
16. DSA UNIT 1 – TREES	10
17. DSA UNIT 2 – GRAPHS.....	12
18. DSA UNIT 3 – DYNAMIC PROGRAMMING.....	13
19. DSA UNIT 4 – BACKTRACKING AND BRANCH AND BOUND	14
20. FACILITY RECOMMENDATION	15
21. DSA TOPIC STATUS SUMMARY	15
22. COMPLEXITY ANALYSIS	16
23. INPUT VALIDATION AND ERROR HANDLING	16
24. TESTING AND TEST CASES.....	17
25. RESULTS AND WORKING SYSTEM	18
26. LIMITATIONS.....	19
27. FUTURE SCOPE.....	19
28. CONCLUSION	20
29. REFERENCES.....	20

1. INTRODUCTION

1.1 Overview

The **AI-Driven Public Facility Utilization System** is a menu-driven console application written in Java. It organises the records of public facilities such as parks, community halls, libraries, sports centers, health centers and recreation centers; connects them in a weighted graph of travel distances; and applies the algorithms of the Data Structures and Algorithms – II (DSA-II) syllabus to search, route, plan, schedule, allocate and recommend.

In this project, “AI-driven” means **algorithmic decision support**: priority-based recommendation with a heap, dynamic-programming optimisation, backtracking search and branch-and-bound pruning. The system does not use a trained machine-learning model and it works on a simulated sample dataset of six facilities; it does not read live occupancy from sensors or booking systems.

1.2 Background

Cities maintain many public facilities, but citizens usually cannot see which facility has free space right now. As a result some places become overcrowded while others stay under-used. Checking hundreds of facilities by hand is not realistic, and facilities are related to each other: they share zones, they are connected by roads of different lengths, and they offer overlapping amenities. A useful system therefore has to store records, understand relationships between facilities, and support planning decisions.

1.3 Problem Statement

To build a system that stores facility records in a searchable form, tracks capacity and current occupancy, represents the road connections between facilities, finds nearby facilities and shortest travel routes, plans maintenance within a limited budget, allocates limited slots, schedules clash-free time slots, matches demand against capacity, optimises improvement tasks, and recommends the facility with the most free capacity.

2. MOTIVATION AND NEED

Manual tracking does not scale with the number of facilities and citizens, and a plain list of facilities does not show how they are connected. Three needs motivated the design:

- **Visibility:** a citizen or manager should be able to see capacity, occupancy and free space of every facility quickly.
- **Balanced utilisation:** demand should be steered toward facilities that have free capacity so that crowding is reduced.

- **Better planning:** maintenance money, time slots and limited capacity should be spent where they give the most benefit.

Alignment with SDG 11 – Sustainable Cities and Communities

Better use of existing public facilities reduces unnecessary travel, reduces crowding and helps municipal planners decide where to invest next.

3. OBJECTIVES

No.	Objective	Realised in
1	Store facility records (ID, name, zone, capacity, occupancy, amenities, hours) and allow add, view and search.	Sections 12, 15
2	Organise records with tree structures (BST, AVL, threaded tree) and browse them in different orders.	Section 16
3	Represent facilities and distances as a weighted graph and find nearby facilities, zones and shortest routes.	Section 17
4	Create a minimum-cost connection network between facilities.	Section 17.5
5	Plan maintenance budget and allocate limited slots with dynamic programming.	Section 18
6	Schedule clash-free time slots, match demand with capacity and optimise tasks with backtracking and branch and bound.	Section 19
7	Recommend the best available facilities for a required capacity.	Section 20
8	Document which DSA-II topics are used and which are not, with reasons.	Section 21

4. SCOPE

Area	Included	Not included
Records	Six sample facilities, add, view, search by ID, three browsing orders	Database storage, file persistence, editing or deleting a facility
Graph	Weighted undirected graph of 9 road connections; BFS, DFS, Dijkstra, Kruskal	Real map data, live traffic, adjacency matrix, Prim's algorithm
Optimisation	0/1 knapsack (maintenance, capacity), graph colouring, sum of subsets, branch and bound	LCS, matrix chain multiplication, travelling salesman
Recommendation	Top three facilities by free capacity using a max-priority queue	Machine-learning prediction of demand; user preference learning
Interface	Console menu with 15 options	Graphical or web interface; user login
Data	Hard-coded sample data plus facilities added at run time	Live occupancy sensors, booking systems

5. EXISTING SYSTEM

Many public facilities are tracked with paper registers, phone calls or simple spreadsheets, and booking systems usually show only one facility at a time.

Limitation of manual / traditional tracking	How this system responds
Citizens cannot see free capacity across facilities	Every record shows capacity, occupancy and the computed Available value; recommendation lists the top three.
Records are scanned one by one	HashMap gives O(1) average lookup by ID; tree structures give ordered browsing.
Relationships between facilities are ignored	A weighted graph stores road distances; BFS, DFS and Dijkstra answer “nearby” and “shortest route”.
Maintenance and capacity decisions are made by guess	Knapsack-style dynamic programming picks the best combination within a budget.
Booking clashes are found only after they happen	Graph colouring with backtracking assigns clash-free time slots.

6. IMPLEMENTED SYSTEM AND KEY FEATURES

6.1 System Overview

When the program starts, loadData() creates six sample facilities, stores each one in an ArrayList, a HashMap, a plain BST, an AVL tree and a threaded tree, and builds the graph with nine weighted edges. A do-while loop then shows the main menu until the user enters 0. Each menu option calls one static method of class Main, reads its inputs with Scanner and prints the result.

6.2 Key Features

Feature	Description	Method / structure
Facility records	Add, view and search facilities by ID	addFacility, viewFacilities, searchFacility; ArrayList + HashMap
Record browsing	ID-wise list, hierarchy, threaded browsing	browseRecords; FacilityTree, ThreadedTree
Facility connections	Show adjacency list with distances	Graph.display
Nearby facilities	Level-by-level traversal from a facility	nearbyFacilities; BFS with Queue
Zone exploration	Depth-first exploration of connected facilities	exploreFacilities; recursive DFS
Shortest travel route	Distance from one facility to all others	shortestRoute; Dijkstra with MinHeap
Minimum network	Cheapest set of roads connecting all facilities	minimumNetwork; Kruskal with DSU
Maintenance planning	Best benefit within a budget	maintenancePlanning; 1-D 0/1 knapsack
Capacity allocation	Best utilisation value for limited slots	capacityAllocation; 2-D 0/1 knapsack
Time-slot scheduling	Clash-free slot per facility	timeScheduling; graph colouring (backtracking)
Demand matching	Subset of demands equal to a target	demandMatching; sum of subsets
Smart optimisation	Best task combination with pruning	smartOptimization; branch and bound
Recommendation	Top three facilities by free capacity	recommendFacility; PriorityQueue (max-heap)

7. TARGET USERS

User	Purpose	Capabilities in the system
Citizen	Find a facility with free space nearby	Search a facility, view availability, find nearby facilities and shortest routes, get recommendations
Facility Manager	Track usage and plan upkeep	Add facilities, browse records, plan maintenance, allocate slots, schedule time slots
Municipal Administrator	Plan city-wide resources	Minimum connection network, demand-capacity matching, smart resource optimisation

The console program has no login, so these roles describe who the features are meant for; the program does not restrict any menu option by user type.

8. FUNCTIONAL REQUIREMENTS

ID	Requirement	Implemented in
FR-01	Add a facility and reject a duplicate ID.	addFacility (records.containsKey)
FR-02	Display all facilities with computed available capacity.	viewFacilities; Facility.toString
FR-03	Search a facility by ID.	searchFacility; HashMap.get
FR-04	List records in ID order, hierarchy order and threaded order.	browseRecords
FR-05	Display facility connections with distances.	Graph.display
FR-06	List facilities reachable from a start facility, level by level.	nearbyFacilities
FR-07	Explore all facilities connected to a start facility.	exploreZone
FR-08	Compute shortest distance from a source to every facility.	shortestRoute
FR-09	Build a minimum-distance connection network and report its total.	minimumNetwork
FR-10	Compute the maximum usage benefit within a maintenance budget.	maintenancePlanning
FR-11	Compute maximum utilisation value for a number of slots.	capacityAllocation
FR-12	Assign one time slot to each facility so connected facilities never share a slot.	assignSlots, safeSlot
FR-13	Find demand values that add up to a target capacity.	subset
FR-14	Find the best task combination for given resources with pruning.	optimize, upperBound
FR-15	Recommend up to three facilities that have the required free capacity.	recommendFacility

9. NON-FUNCTIONAL REQUIREMENTS

Category	Requirement	How it is met
Performance	Responses should be immediate for small and medium inputs.	Algorithms use efficient structures: HashMap lookup, heap-based Dijkstra, DP tables; sample graph has 6 vertices.
Usability	Simple text menu, readable results.	Numbered menu, prompts for every input, section headings in every output.
Maintainability	Clear separation of data classes and algorithms.	Separate classes for Facility, Edge, Graph, trees, MinHeap and DSU; one static method per feature.
Portability	Runs anywhere Java runs.	Uses only java.util; no external libraries.

Category	Requirement	How it is met
Correctness	Algorithms must follow their definitions.	Standard recurrences and invariants; outputs were checked by running every menu option (Section 24).
Reliability	Handle common “not found” cases.	Duplicate ID, missing ID, no match, no schedule and no suitable facility all print a message (Section 23).

10. SYSTEM ARCHITECTURE

The program follows a layered structure. The user works through a console menu; class Main acts as the controller; three groups of components do the work (record layer, graph layer and optimisation layer); all of them rely on the Facility data model.

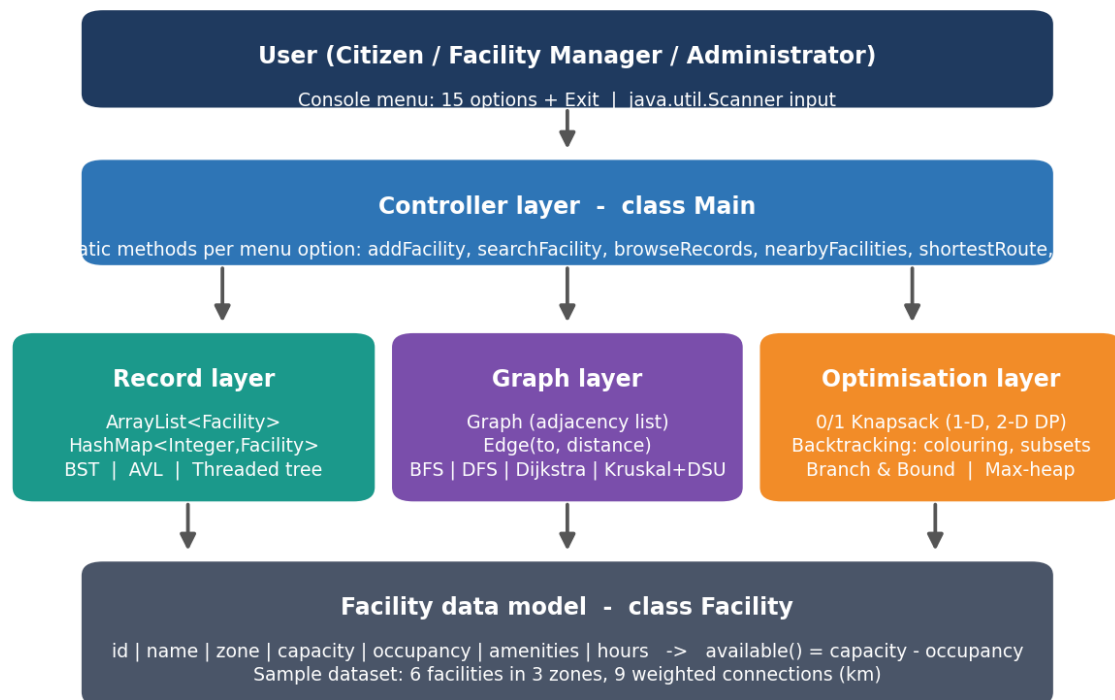


Figure 10.1: Layered architecture of the AI-Driven Public Facility Utilization System

Figure 10.1: Layered architecture of the implemented system

10.1 Layer Responsibilities

Layer	Responsibility	Classes / structures
Interface	Show the menu, read choices and inputs	Scanner, do-while loop, switch
Controller	Map each menu option to one operation	class Main (static methods)
Record layer	Store and order facility records	ArrayList, HashMap, FacilityTree, FacilityAVL, ThreadedTree
Graph layer	Connections, traversal, shortest paths, spanning tree	Graph, Edge, MinHeap, DSU
Optimisation layer	Planning, scheduling and matching	DP arrays, recursion, Task, PriorityQueue
Data model	One facility record	class Facility

10.2 Request Flow

User enters a menu number → switch in main() calls the matching static method → the method reads extra inputs → the method uses the required structure (HashMap, tree, graph or DP table) → the result is printed → the menu appears again.

11. TECHNOLOGY STACK

Component	Technology	Use in the project
Language	Java (lambdas used, so Java 8 or later; tested on OpenJDK 21)	Complete implementation
Collections	ArrayList, HashMap, PriorityQueue, LinkedList / Queue	Records, lookup, heaps, BFS queue
Input	java.util.Scanner	Console input
Utilities	Arrays, Comparator, Math	Sorting edges and tasks, DP maximum
External libraries	None	Only the Java standard library is used
Version control	GitHub	Source code and sample data (link in Appendix A)

12. DATA DESIGN

12.1 Facility Record

Field	Type	Description
id	int	Unique facility identifier (key for lookup and tree ordering)
name	String	Facility name
zone	String	Zone label (Zone A, Zone B, Zone C)
capacity	int	Maximum number of people the facility can hold
occupancy	int	Current number of people
amenities	String	Comma-separated list of amenities
hours	String	Operating hours
available()	method	capacity – occupancy; used by recommendation

12.2 Sample Dataset

ID	Facility	Zone	Capacity	Occupancy	Available	Used %	Hours
0	Central Park	A	100	60	40	60	6 AM – 8 PM
1	Community Hall	A	80	30	50	37	9 AM – 9 PM
2	Public Library	B	120	90	30	75	8 AM – 8 PM
3	Sports Center	B	150	100	50	66	6 AM – 10 PM
4	Health Center	C	70	45	25	64	8 AM – 6 PM
5	Recreation Center	C	90	40	50	44	9 AM – 9 PM

“Used %” is $\text{occupancy} \times 100 / \text{capacity}$ using integer division, exactly as computed in `capacityAllocation()`.

12.3 Graph Data

Edge	Distance (km)	Edge	Distance (km)	Edge	Distance (km)
0 – 1	2	2 – 3	2	3 – 5	4
0 – 2	4	2 – 4	6	4 – 5	2
1 – 2	1	3 – 4	3	1 – 3	5

The graph is undirected: `Graph.addEdge(a, b, distance)` adds an `Edge` object to the list of both vertices. It is stored as an `ArrayList` of `ArrayLists` (adjacency list), so space is $\Theta(V + E)$.

13. CLASS AND MODULE DESIGN

Class	Role	Key members
Facility	Data model for one record	id, name, zone, capacity, occupancy, amenities, hours; <code>available()</code> ; <code>toString()</code>
Edge	One road connection	<code>to</code> , <code>distance</code>
Graph	Adjacency list	<code>list</code> ; <code>addEdge()</code> ; <code>display()</code>
TreeNode / FacilityTree	Plain binary search tree on id	<code>insert</code> , <code>inorder</code> , <code>preorder</code> , <code>postorder</code>
AVLNode / FacilityAVL	Self-balancing tree on id	<code>height</code> , <code>balance</code> , <code>rightRotate</code> , <code>leftRotate</code> , <code>insert</code> , <code>display</code>
ThreadNode / ThreadedTree	Threaded binary tree	<code>insert</code> , <code>makeThreads</code> , <code>display</code> (stackless <code>inorder</code>)
MinHeap	Wrapper over <code>PriorityQueue<int[]></code>	<code>add(node, distance)</code> , <code>remove()</code> , <code>empty()</code>
DSU	Disjoint set (union-find)	<code>find</code> with path compression, <code>union</code>
Main.Task	Improvement task	<code>cost</code> , <code>value</code>
Main	Controller and algorithms	<code>static data</code> , 15 feature methods, <code>loadData()</code> , <code>main()</code>

14. CONSOLE INTERFACE AND MENU NAVIGATION

The interface is a text menu titled “AI-DRIVEN PUBLIC FACILITY UTILIZATION SYSTEM”. The user types a number and presses Enter.

No.	Menu option	Method	DSA concept
1	View Public Facilities	<code>viewFacilities</code>	List traversal
2	Add New Facility	<code>addFacility</code>	HashMap, BST, AVL, threaded tree insert
3	Search Facility	<code>searchFacility</code>	HashMap lookup
4	Browse Facility Records	<code>browseRecords</code>	Tree traversals, threaded tree
5	View Facility Connections	<code>Graph.display</code>	Adjacency list
6	Find Nearby Facilities	<code>nearbyFacilities</code>	BFS
7	Explore Facility Zones	<code>exploreFacilities</code>	DFS
8	Find Shortest Travel Route	<code>shortestRoute</code>	Dijkstra, min-heap
9	Create Minimum Facility Network	<code>minimumNetwork</code>	Kruskal, union-find
10	Maintenance Budget Planning	<code>maintenancePlanning</code>	0/1 knapsack (1-D)

No.	Menu option	Method	DSA concept
11	Facility Capacity Allocation	capacityAllocation	0/1 knapsack (2-D)
12	Facility Time-Slot Scheduling	timeScheduling	Graph colouring, backtracking
13	Demand-Capacity Matching	demandMatching	Sum of subsets, backtracking
14	Smart Resource Optimization	smartOptimization	Branch and bound
15	Recommend Best Facility	recommendFacility	Priority queue (max-heap)
0	Exit	—	Loop termination

Unknown menu numbers print “Invalid choice.” and the menu is shown again.

15. FACILITY RECORD MANAGEMENT

Three menu options manage the records: **Add** (option 2), **View** (option 1) and **Search** (option 3).

15.1 Adding a Facility

- The ID is read first. If `records.containsKey(id)` is true, the method prints “Facility already exists.” and returns, so IDs are unique.
- Name, zone, capacity, current occupancy, amenities and operating hours are read; a Facility object is created.
- The object is added to the ArrayList (display order), the HashMap (fast lookup) and the three trees (BST, AVL, threaded tree).

15.2 Searching

`searchFacility()` calls `records.get(id)`. HashMap gives $O(1)$ average time, compared with $O(n)$ for scanning the list. If the result is null, “Facility not found.” is printed.

Observation from testing

A facility added through option 2 is stored in the record structures but is **not** added to the graph, which is built once for the six sample facilities. Graph-based options (6–9, 12) therefore cover only the original six, and using a new ID as the start facility in option 6 raises `ArrayIndexOutOfBoundsException` (Test TC-24).

16. DSA UNIT 1 – TREES

Facility records are also kept in tree structures keyed by facility ID. Each tree is built on every insertion; the table shows where each one is used.

16.1 Binary Search Tree (FacilityTree)

Smaller IDs go to the left subtree and larger IDs to the right. Three traversals are provided: **inorder** (left, node, right) gives ascending ID order and is used by “ID-wise Facility List”; **preorder** (node, left, right) is used by “Facility Hierarchy”; **postorder** is implemented but not offered in the menu.

FacilityTree.insert

```
TreeNode insert(TreeNode node, Facility f) {
    if (node == null) return new TreeNode(f);
    if (f.id < node.data.id) node.left = insert(node.left, f);
    else node.right = insert(node.right, f);
    return node;
}
```

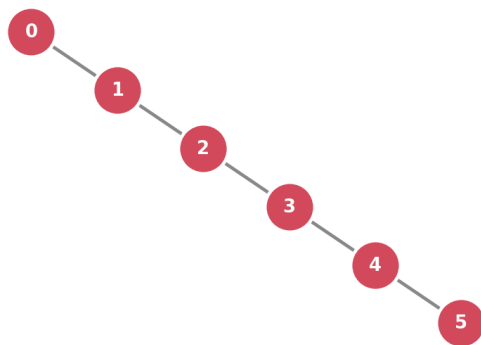
Skewed tree

The sample facilities are inserted with IDs 0, 1, 2, 3, 4, 5 in ascending order, so the plain BST becomes a chain of height 6. This is why the “Facility Hierarchy” (preorder) output is identical to the ID-wise (inorder) output. Search in such a tree costs $O(n)$, which is the worst case in Section 22.

16.2 AVL Tree (FacilityAVL)

The AVL tree keeps the balance factor (height of left – height of right) of every node in $\{-1, 0, 1\}$. After an insertion it updates heights and applies one of four rotations: **LL** (right rotation), **RR** (left rotation), **LR** (left then right) and **RL** (right then left). Inserting 0–5 gives a balanced tree of height 3 with root 3.

(a) Plain BST: IDs 0-5 inserted in order -> skewed, height 6



(b) AVL tree: same IDs -> balanced, height 3

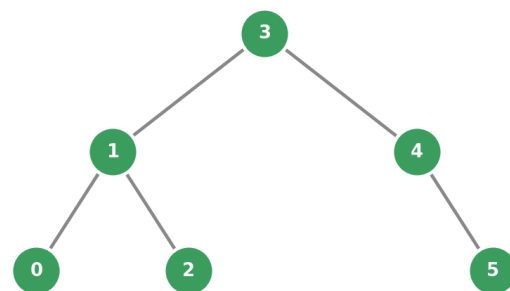


Figure 16.1: Same six IDs in a plain BST (skewed) and in the AVL tree (balanced)

Honest note on use

The AVL tree is built and balanced on every insertion, but the menu never queries or prints it (FacilityAVL.display() is not called). It is therefore a demonstration of the balancing technique and the intended balanced index; the lookup in option 3 is served by the HashMap.

16.3 Threaded Binary Tree (ThreadedTree)

In a threaded tree a null right link is replaced by a **thread** to the inorder successor, so inorder traversal needs no recursion or stack. makeThreads() walks the tree inorder, remembers the previous node and, if its right link is null, points it to the current node and sets thread = true. display() starts at the leftmost node and follows threads. This is “Facility Record Browsing” (browse option 3). Traversal is $O(n)$ time with $O(1)$ extra space.

ThreadedTree.makeThreads

```
void makeThreads(ThreadNode node, ThreadNode[] previous) {
    if (node == null) return;
    makeThreads(node.left, previous);
    if (previous[0] != null && previous[0].right == null) {
        previous[0].right = node; previous[0].thread = true;
    }
    previous[0] = node;
    if (!node.thread) makeThreads(node.right, previous);
}
```

16.4 Heaps and Priority Queue

Java's PriorityQueue (a binary heap stored in an array) is used in two places: a **min-heap** (MinHeap class) for Dijkstra's algorithm and a **max-heap** ordered by available() for facility recommendation. Insert and remove are $O(\log n)$; peek is $O(1)$.

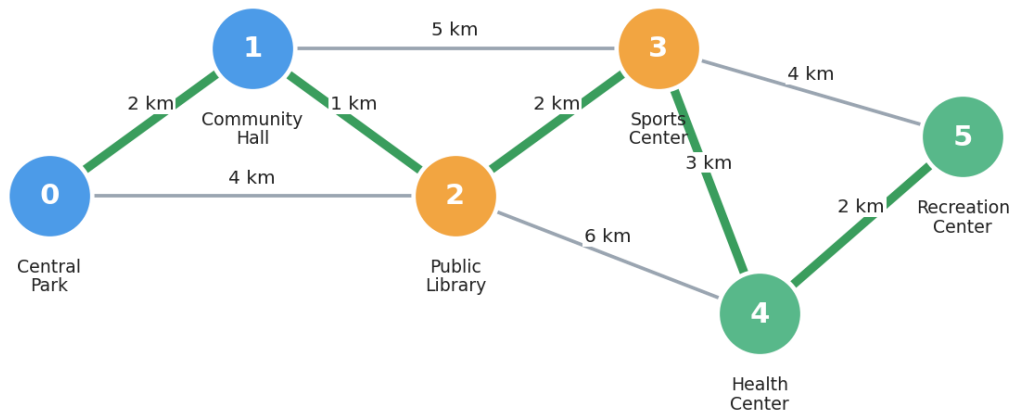
Tree concept	Where used	Menu option
BST insert and inorder / preorder	FacilityTree	4 → 1, 4 → 2
BST postorder	FacilityTree.postorder (implemented)	Not in menu
AVL insert with rotations	FacilityAVL	Built on add; not queried
Threaded tree traversal	ThreadedTree	4 → 3
Heap / priority queue	MinHeap (Dijkstra), recommendation queue	8, 15

17. DSA UNIT 2 – GRAPHS

17.1 Graph Representation

Facilities are vertices and roads are undirected edges weighted by distance in kilometres. An **adjacency list** is used because the network is sparse ($V = 6$, $E = 9$ against 15 possible pairs). The adjacency matrix would need $\Theta(V^2)$ space and is not used.

(a) Facility network - node colour = zone; green = Kruskal MST (10 km)



(b) Time-slot colouring with 3 slots (red = slot 1, blue = slot 2, green = slot 3)

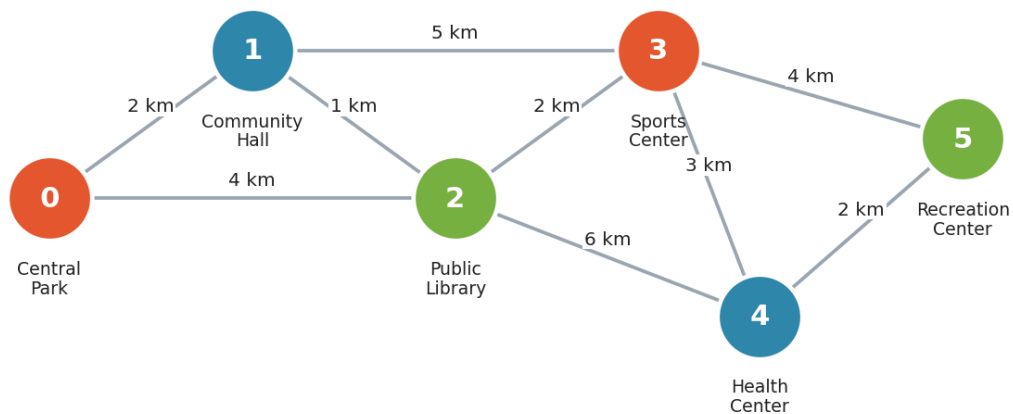


Figure 17.1: Facility network, minimum spanning tree (a) and time-slot colouring (b)

17.2 Breadth-First Search – Nearby Facilities

`nearbyFacilities()` starts at the chosen facility, marks it visited and uses a Queue. It repeatedly removes a facility, prints it and adds every unvisited neighbour. Facilities are therefore listed level by level (fewest roads first). Time $O(V + E)$, space $O(V)$. Note that the method lists all reachable facilities in BFS order; it does not stop after k hops.

17.3 Depth-First Search – Zone Exploration

`exploreZone()` is recursive: it marks the facility visited, prints it and recurses into each unvisited neighbour, going as deep as possible before backtracking. Time $O(V + E)$, space $O(V)$ for the recursion stack. It shows the whole connected cluster of the start facility.

17.4 Dijkstra's Algorithm – Shortest Route

`shortestRoute()` keeps a distance array initialised to `Integer.MAX_VALUE`, sets the source distance to 0 and uses the MinHeap ordered by distance. For every popped facility u and edge (u, v, w) it relaxes: if $\text{distance}[u] + w < \text{distance}[v]$, the distance is updated and $(v, \text{distance}[v])$ is pushed. Edge weights are non-negative, which Dijkstra requires. Time $O((V + E) \log V)$.

Facility	0	1	2	3	4	5
Distance from 0 (km)	0	2	3	5	8	9

Facility	0	1	2	3	4	5
Best path	0	0→1	0→1→2	0→1→2→3	0→1→2→3→4	0→1→2→3→5

For example, facility 2 is reached in 3 km through facility 1 (2 + 1), which is shorter than the direct 4 km road.

17.5 Kruskal's Algorithm – Minimum Connection Network

minimumNetwork() collects each undirected edge once ($i < to$), sorts the edges by distance and scans them with a **disjoint-set union (DSU)** that uses path compression. An edge is accepted only if its end points are in different sets, so no cycle is created. Time $O(E \log E)$.

Sorted edge	Distance	Decision	Reason
1 – 2	1 km	USED	Different sets → joined
0 – 1	2 km	USED	Different sets → joined
2 – 3	2 km	USED	Different sets → joined
4 – 5	2 km	USED	Different sets → joined
3 – 4	3 km	USED	Joins {0,1,2,3} and {4,5}
0 – 2, 3 – 5, 1 – 3, 2 – 4	4, 4, 5, 6 km	NOT USED	Would create a cycle

Total minimum connection distance = 1 + 2 + 2 + 2 + 3 = **10 km** for five edges ($V - 1$).

18. DSA UNIT 3 – DYNAMIC PROGRAMMING

Dynamic programming stores answers to overlapping sub-problems in a table so that each is computed once. Both DP features in this project are 0/1 knapsack problems: every task or facility is either chosen or not chosen.

18.1 Maintenance Budget Planning (1-D knapsack)

Each task has a cost (or time) and a usage-importance value. For a budget B , $dp[j]$ is the best benefit with budget j . The array is updated from high j to low j so that each task is used at most once:

Recurrence in maintenancePlanning

```
for (int i = 0; i < n; i++)
    for (int j = budget; j >= cost[i]; j--)
        dp[j] = Math.max(dp[j], benefit[i] + dp[j - cost[i]]);
```

Time $O(n \cdot B)$; space $O(B)$ because only one row is kept (the space optimisation mentioned in Progress Report II). Plain recursion would take $O(2^n)$.

Task	Cost / time	Importance
A	3	40
B	4	50
C	2	30
D	5	60

Budget 9: the best choice is A + B + C (3 + 4 + 2 = 9) with benefit 40 + 50 + 30 = **120**; the program printed "Maximum Usage Benefit = 120".

18.2 Facility Capacity Allocation (2-D knapsack)

Every facility becomes an item: **need** = free capacity (at least 1) and **benefit** = utilisation percentage = occupancy $\times$ 100 / capacity. For S available slots, $dp[i][s]$ is the best total benefit using the first i facilities and s slots; either facility i is skipped ($dp[i-1][s]$) or taken ($benefit + dp[i-1][s-need]$). Time and space $\Theta(n \cdot S)$.

Facility	0	1	2	3	4	5
Need (free capacity)	40	50	30	50	25	50
Benefit (used %)	60	37	75	66	64	44

With 60 slots the best selection is facility 2 (need 30, benefit 75) plus facility 4 (need 25, benefit 64): 55 slots and benefit **139**, which matches the program output.

19. DSA UNIT 4 – BACKTRACKING AND BRANCH AND BOUND

19.1 Time-Slot Scheduling (Graph Colouring)

Facilities that are connected by a road must not use the same time slot. `assignSlots()` tries slots 1 ... m for facility 0, 1, 2 ... in order. `safeSlot()` checks that no neighbour already holds the slot. If the next facility cannot be placed, the assignment is reset to 0 (**backtracking**) and the next slot is tried. Worst case $O(m^V)$.

Slots available	Result
3	Facility 0→1, 1→2, 2→3, 3→1, 4→2, 5→3 (valid schedule)
2	"No valid time-slot schedule." – facilities 0, 1 and 2 are mutually connected (a triangle) and need three different slots

Progress Report II describes bookings as vertices; in the implemented program the vertices are the facilities themselves.

19.2 Demand-Capacity Matching (Sum of Subsets)

`subset()` decides for each demand value whether to include it. If the remaining target becomes 0 a match is printed; if it becomes negative or the values are exhausted, the method returns false and the last chosen value is removed (backtracking). Worst case $O(2^n)$. The first solution found is reported, not all solutions. For demands 10, 25, 15, 30, 20 and target 60 the result is [10, 30, 20].

19.3 Smart Resource Optimization (Branch and Bound)

Tasks (cost, benefit) are sorted by benefit $\div$ cost, highest first. `optimize()` explores include / exclude choices. Before exploring a branch, `upperBound()` estimates the best value still reachable by filling the remaining capacity greedily and taking a **fraction** of the last task. If this bound is not better than the best complete solution found so far, the whole branch is **pruned**.

```
Branch and bound in optimize()
if (upperBound(tasks, index, capacity, value) <= best)
    return; // prune this branch
if (t.cost <= capacity)
    optimize(tasks, index + 1, capacity - t.cost, value + t.value); // include
    optimize(tasks, index + 1, capacity, value); // exclude
```

For tasks (10, 60), (20, 100), (30, 120), (15, 70), (5, 20) and 50 units of resource, the best selection is the tasks with costs 10, 15, 5 and 20 (total cost 50) giving **250**, as printed by the program. Worst case $O(2^n)$, but pruning removes many branches in practice.

20. FACILITY RECOMMENDATION

`recommendFacility()` reads the required free capacity, adds every facility with `available() ≥ required` to a `PriorityQueue` ordered by `available()` in descending order (a max-heap) and prints up to three facilities from the top. Building the queue costs $O(n \log n)$; each poll is $O(\log n)$. If the queue is empty the message “No suitable facility available.” is shown.

For a required capacity of 40, facilities 1, 3 and 5 (all with 50 free) are returned and facility 0 (40 free) is not shown because only three are printed; ties are returned in the heap’s internal order.

21. DSA TOPIC STATUS SUMMARY

Following the feedback of Progress Report I, only topics that genuinely help the facility problem are applied; every other topic is marked NOT USED with a reason.

Unit	Topic	Status	Reason / where
1	Binary search tree + traversals	USED	Ordered listing and hierarchy (option 4)
1	AVL tree	PARTIAL	Built on every add; not queried by any option
1	Threaded binary tree	USED	Stackless record browsing (option 4 → 3)
1	Heap / priority queue	USED	Dijkstra min-heap; recommendation max-heap
2	Graph, adjacency list	USED	Facility network
2	Adjacency matrix	NOT USED	Sparse graph; list is smaller
2	BFS	USED	Nearby facilities
2	DFS	USED	Zone exploration
2	Dijkstra	USED	Shortest travel distance
2	Kruskal spanning tree	USED	Minimum connection network
2	Prim’s algorithm	NOT USED	Kruskal already solves the problem
3	0/1 knapsack	USED	Maintenance planning
3	Resource allocation	USED	Capacity allocation (knapsack form)
3	LCS	NOT USED	Facility data is attribute-based, no sequences
3	Matrix chain multiplication	NOT USED	No matrix products in the system
4	Graph colouring (backtracking)	USED	Time-slot scheduling
4	Sum of subsets (backtracking)	USED	Demand-capacity matching
4	Branch and bound	USED	Smart resource optimisation
4	Travelling salesman	NOT USED	No need to visit every facility in one tour
5	Red-Black, B-tree, B+ tree, binomial and Fibonacci heaps	NOT USED	Not required by the problem

22. COMPLEXITY ANALYSIS

Notation: n = records or items, h = tree height, V and E = graph vertices and edges, B = budget, S = slots, m = number of colours. Values are theoretical.

Operation	Best	Average	Worst	Space
HashMap search by ID	$O(1)$	$O(1)$	$O(n)$	$O(n)$
BST insert / search	$O(1)$	$O(\log n)$	$O(n)$ – happens here	$O(n)$
AVL insert / search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
Tree traversal (in, pre, post)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(h)$
Threaded traversal / makeThreads	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(1) / O(h)$
PriorityQueue add / poll	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
BFS, DFS	$O(1)$	$O(V+E)$	$O(V+E)$	$O(V)$
Dijkstra (binary heap)	—	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V+E)$
Kruskal (sort + DSU)	—	$O(E \log E)$	$O(E \log E)$	$O(V+E)$
Maintenance knapsack (1-D)	$\Theta(nB)$	$\Theta(nB)$	$\Theta(nB)$	$\Theta(B)$
Capacity allocation (2-D)	$\Theta(nS)$	$\Theta(nS)$	$\Theta(nS)$	$\Theta(nS)$
Graph colouring	—	—	$O(m^V)$	$O(V)$
Sum of subsets	—	—	$O(2^n)$	$O(n)$
Branch and bound	—	pruned	$O(2^n)$	$O(n)$
Recommendation	—	$O(n \log n)$	$O(n \log n)$	$O(n)$

The Dijkstra implementation does not skip outdated heap entries, so the heap can hold up to E entries; this keeps the bound at $O(E \log V)$. No running-time benchmark was measured.

23. INPUT VALIDATION AND ERROR HANDLING

23.1 Situations Handled

Situation	Behaviour
Duplicate facility ID	“Facility already exists.” – record is not added
Facility ID not found in search	“Facility not found.”
Unknown menu number	“Invalid choice.” and the menu is shown again
No subset reaches the target	“No matching combination found.”
Not enough time slots	“No valid time-slot schedule.”
No facility with enough free capacity	“No suitable facility available.”

23.2 Situations Not Handled

- Non-numeric input where a number is expected makes `Scanner.nextInt()` throw `InputMismatchException` and the program stops (by code inspection; not executed).
- Facility IDs outside 0 ... 5 in options 6, 7 and 8, and IDs of newly added facilities, cause `ArrayIndexOutOfBoundsException`.
- Occupancy larger than capacity, negative capacity or negative costs are accepted without checking.

- In option 8, facilities that cannot be reached would print Integer.MAX_VALUE as the distance (cannot happen with the connected sample graph).

24. TESTING AND TEST CASES

Every menu option was run by feeding scripted input to the program on OpenJDK 21 (source-file mode). The results below are the real outputs of those runs. No automated unit-test framework and no performance test was used.

ID	Scenario	Input	Observed result	Status
TC-01	View facilities	Option 1	Six records with correct Available values	PASSED
TC-02	Search existing ID	Option 3, ID 2	Public Library details shown	PASSED
TC-03	Search missing ID	Option 3, ID 9	"Facility not found."	PASSED
TC-04	Add duplicate ID	Option 2, ID 0	"Facility already exists."	PASSED
TC-05	Add new facility	ID 6, City Museum	Added; appears in view (7 records)	PASSED
TC-06	ID-wise list	Option 4 → 1	IDs 0 to 5 in ascending order	PASSED
TC-07	Hierarchy (preorder)	Option 4 → 2	Same order as TC-06 (skewed BST)	PASSED
TC-08	Threaded browsing	Option 4 → 3	IDs 0 to 5 in order	PASSED
TC-09	Show connections	Option 5	Adjacency list of 6 facilities	PASSED
TC-10	Nearby (BFS)	Option 6, start 0	0, 1, 2, 3, 4, 5	PASSED
TC-11	Zone exploration (DFS)	Option 7, start 0	0, 1, 2, 3, 4, 5	PASSED
TC-12	Shortest route	Option 8, source 0	0, 2, 3, 5, 8, 9 km	PASSED
TC-13	Minimum network	Option 9	5 edges, total 10 km	PASSED
TC-14	Maintenance planning	4 tasks, budget 9	Benefit 120	PASSED
TC-15	Capacity allocation	60 slots	Utilisation value 139	PASSED
TC-16	Scheduling, 3 slots	Option 12, 3	Slots 1, 2, 3, 1, 2, 3	PASSED
TC-17	Scheduling, 2 slots	Option 12, 2	"No valid time-slot schedule."	PASSED
TC-18	Demand matching	10, 25, 15, 30, 20; target 60	[10, 30, 20]	PASSED
TC-19	No matching subset	10, 25, 15; target 7	"No matching combination found."	PASSED
TC-20	Branch and bound	5 tasks, resources 50	Best value 250	PASSED
TC-21	Recommend, need 40	Option 15, 40	Facilities 1, 5, 3 (50 free each)	PASSED
TC-22	Recommend, need 200	Option 15, 200	"No suitable facility available."	PASSED
TC-23	Invalid menu choice	99	"Invalid choice."	PASSED
TC-24	BFS from newly added facility	Add ID 6, option 6, start 6	ArrayIndexOutOfBoundsException	KNOWN DEFECT
TC-25	Letters instead of numbers	Any numeric prompt	InputMismatchException expected	NOT RUN

25. RESULTS AND WORKING SYSTEM

The captured output below comes from running the program on the sample dataset.

Option 5 – facility connections

```
--- Public Facility Connections ---
Facility 0 -> 1 (2 km) 2 (4 km)
Facility 1 -> 0 (2 km) 2 (1 km) 3 (5 km)
Facility 2 -> 0 (4 km) 1 (1 km) 3 (2 km) 4 (6 km)
Facility 3 -> 1 (5 km) 2 (2 km) 4 (3 km) 5 (4 km)
Facility 4 -> 2 (6 km) 3 (3 km) 5 (2 km)
Facility 5 -> 3 (4 km) 4 (2 km)
```

Option 8 – Dijkstra from facility 0

```
--- Shortest Travel Routes ---
Facility 0 to Facility 0 = 0 km
Facility 0 to Facility 1 = 2 km
Facility 0 to Facility 2 = 3 km
Facility 0 to Facility 3 = 5 km
Facility 0 to Facility 4 = 8 km
Facility 0 to Facility 5 = 9 km
```

Option 9 – Kruskal minimum network

```
--- Minimum Facility Connection ---
Facility 1 - Facility 2 : 1 km
Facility 0 - Facility 1 : 2 km
Facility 2 - Facility 3 : 2 km
Facility 4 - Facility 5 : 2 km
Facility 3 - Facility 4 : 3 km
Minimum Connection Distance = 10 km
```

Option 12 – scheduling with 3 slots

```
--- Facility Time-Slot Schedule ---
Facility 0 -> Time Slot 1
Facility 1 -> Time Slot 2
Facility 2 -> Time Slot 3
Facility 3 -> Time Slot 1
Facility 4 -> Time Slot 2
Facility 5 -> Time Slot 3
```

Option 15 – required capacity 40

```
--- Recommended Public Facilities ---
ID: 1 | Community Hall | Zone: Zone A | Capacity: 80 | Occupancy: 30 | Available: 50 | ...
ID: 5 | Recreation Center | Zone: Zone C | Capacity: 90 | Occupancy: 40 | Available: 50 | ...
ID: 3 | Sports Center | Zone: Zone B | Capacity: 150 | Occupancy: 100 | Available: 50 | ...
```

25.1 Summary of Results

Capability	Result on sample data
Nearby search and zone exploration	All six facilities reached from facility 0
Shortest travel	Farthest facility (5) is 9 km from facility 0 via 1, 2 and 3
Network cost	All facilities can be connected with 10 km of road instead of 26 km in total
Maintenance planning	Benefit 120 within budget 9
Capacity allocation	Utilisation value 139 with 60 slots
Scheduling	Three time slots are sufficient and two are not
Optimisation	Branch and bound found value 250 for resources 50

The sum of all nine road distances is $2 + 4 + 1 + 5 + 2 + 6 + 3 + 4 + 2 = 26$ km, which is the figure compared with the 10 km minimum network.

26. LIMITATIONS

- “AI-driven” is limited to heuristic and optimisation algorithms; there is no machine-learning model and no demand prediction.
- Data are simulated and hard-coded; there is no persistence, so facilities added at run time are lost when the program ends.
- New facilities are not connected to the graph, so graph features cover only the six sample facilities and a new ID as a start vertex crashes option 6.
- The plain BST is skewed for ascending IDs, and the AVL tree is built but never queried.
- BFS lists all reachable facilities; a “within k hops” limit is not implemented. Prim’s algorithm is not implemented (Kruskal only).
- Capacity allocation uses utilisation percentage as the benefit, so it favours busier facilities; this is a modelling choice that should be reviewed.
- There are no edit or delete operations, no input validation for non-numeric or negative values, and no automated tests.
- Backtracking and branch and bound are exponential in the worst case and suit small inputs only.

27. FUTURE SCOPE

Area	Enhancement
Data	Store facilities in a file or database; edit and delete records; import real facility data.
Graph	Add new facilities and their roads automatically; support “within k hops” and path printing; add Prim’s algorithm.
Records	Serve ID and range search from the AVL tree or a balanced index; rebalance on deletion.
AI	Predict hourly occupancy from history and recommend using predicted crowding and distance together.
Interface	Replace the console with a web or mobile interface and a map view.
Quality	Validate all input, handle exceptions and add unit tests for every algorithm.

28. CONCLUSION

The AI-Driven Public Facility Utilization System shows how DSA-II concepts solve a practical city-planning problem. Facility records are kept in a HashMap and in tree structures; the road network is a weighted graph processed with BFS, DFS, Dijkstra and Kruskal; dynamic programming plans maintenance and slot allocation; backtracking and branch and bound handle scheduling, matching and task optimisation; and a priority queue recommends the facilities with the most free capacity.

All fifteen menu options were executed on the sample dataset and produced the expected results, with one defect and several limitations recorded honestly in Sections 24 and 26. The project is a prototype on simulated data, and its main value is the clear mapping from each problem requirement to a suitable data structure or algorithm.

29. REFERENCES

- [1] The satisfaction of ecological environment in sports public services by artificial intelligence and big data (2025).
- [2] AI-Driven Optimization of Sanitary Facilities in Office Buildings: A Machine Learning Approach Using LSTM Neural Networks (2025).
- [3] Smart Sport: Crowd Counting Meets Large Language Models for Smart Facility Management (2026).
- [4] Data Structures and Algorithms – II (CCSE0301), Units 1–4 course material and notes of Mr. Shamshad Ali, NIET Greater Noida.
- [5] Oracle, Java Platform Standard Edition API documentation ([java.util collections](#)).